

Performance Considerations

Designing the API to use state objects creates several performance advantages. These include validating state at object creation time, enabling caching of state objects in hardware, and greatly reducing the amount of state that is passed during a state-setting API call (by passing a handle to the state object instead of the state).

To achieve these performance improvements, you should create your state objects when your application starts up, well before your render loop. State objects are immutable, that is, once they are created, you cannot change them. Instead you must destroy and recreate them.

You could create several sampler objects with various sampler-state combinations. Changing the sampler state is then accomplished by calling the appropriate "Set" API which passes a handle to the object (as opposed to the sampler state). This significantly reduces the amount of overhead during each render frame for changing state since the number of calls and the amount of data are greatly reduced.

Alternatively, you can choose to use the effect system which will automatically manage efficient creation and destruction of state objects for your application.

Related topics

[Graphics pipeline](#)

[Views](#)

Views

Article • 10/20/2022

The term "view" is used to mean "data in the required format". For example, a Constant Buffer View (CBV) would be constant buffer data correctly formatted. This section describes the most common and useful views.

In this section

Topic	Description
Constant buffer view (CBV)	Constant buffers contain shader constant data. The value of them is that the data persists, and can be accessed by any GPU shader, until it is necessary to change the data.
Vertex buffer view (VBV) and Index buffer view (IBV)	A vertex buffer holds data for a list of vertices. The data for each vertex can include position, color, normal vector, texture co-ordinates, and so on. An index buffer holds integer indexes (offsets) into a vertex buffer, and is used to define and render an object made up of a subset of the full list of vertices.
Shader resource view (SRV) and Unordered Access view (UAV)	Shader resource views typically wrap textures in a format that the shaders can access them. An unordered access view provides similar functionality, but enables the reading and writing to the texture (or other resource) in any order.
Sampler	Sampling is the process of reading input values from a texture, or other resource. A "sampler" is any object that reads from resources.
Render target view (RTV)	Render targets enable a scene to be rendered to a temporary intermediate buffer, rather than to the back buffer to be rendered to the screen. This feature enables use of the complex scene that might be rendered, perhaps as a reflection texture or other purpose within the graphics pipeline, or perhaps to add additional pixel shader effects to the scene before rendering.

Topic	Description
Depth stencil view (DSV)	A depth stencil view provides the format and buffer for holding depth and stencil information. The depth buffer is used to cull the drawing of pixels that would be invisible to the viewer as they are occluded from view by a closer object. The stencil buffer can be used to cull all drawing outside of a defined shape.
Stream output view (SOV)	Stream output views enable the vertex information that the vertex, tessellation and geometry shaders have come up with to be streamed back out to the application for further use. For example, an object that has been distorted by these shaders could be written back to the application to provide more accurate input to a physics or other engine. In practice though, stream output views are an infrequently used feature of the graphics pipeline.
Rasterizer ordered view (ROV)	Rasterizer ordered views enable some of the limitations of a depth buffer to be addressed, in particular having multiple textures containing transparency all applying to the same pixels.

Related topics

[Direct3D Graphics Learning Guide](#)

Constant buffer view (CBV)

Article • 10/20/2022

Constant buffers contain shader constant data. The value of them is that the data persists, and can be accessed by any GPU shader, until it is necessary to change the data.

Typical data for a constant buffer would be world, projection and view matrices, which remain constant throughout the drawing of one frame.

Constant buffer layout should match the HLSL layout (refer to [Packing Rules for Constant Variables](#)).

Related topics

[Views](#)

Vertex buffer view (VBV) and Index buffer view (IBV)

Article • 10/20/2022

A vertex buffer holds data for a list of vertices. The data for each vertex can include position, color, normal vector, texture co-ordinates, and so on. An index buffer holds integer indexes (offsets) into a vertex buffer, and is used to define and render an object made up of a subset of the full list of vertices.

The definition of a single vertex is often up to the application to define, such as:

syntax

```
struct CUSTOMVERTEX {  
    FLOAT x, y, z;        // The position  
    FLOAT nx, ny, nz;     // The normal  
    DWORD color;          // RGBA color  
    FLOAT tu, tv;         // The texture coordinates.  
};
```

The definition of CUSTOMVERTEX would then be passed to the graphics driver when creating vertex buffers.

Related topics

[Views](#)

Shader resource view (SRV) and Unordered Access view (UAV)

Article • 10/20/2022

Shader resource views typically wrap textures in a format that the shaders can access them. An unordered access view provides similar functionality, but enables the reading and writing to the texture (or other resource) in any order.

Wrapping a single texture is probably the simplest form of shader resource view. More complex examples would be a collection of subresources (individual arrays, planes, or colors from a mipmapped texture), 3D textures, 1D texture color gradients, and so on.

Unordered access views are slightly more costly in terms of performance, but allow, for example, a texture to be written to at the same time that it is being read from. This enables the updated texture to be re-used by the graphics pipeline for some other purpose. Shader resource views are for read only use (which is the most common use of resources).

Related topics

[Views](#)

Sampler

Article • 10/20/2022

Sampling is the process of reading input values from a texture, or other resource. A "sampler" is any object that reads from resources.

There are many issues and artifacts from sampling from a texture and rendering to a screen area. For example, if the area to be rendered to is 50 by 50 pixels, and the texture is 16 by 16 pixels or 256 by 256 pixels, then some considerable stretching or shrinking of the texture needs to be applied. Because this mismatch in size leads to unwelcome artifacts, filtering techniques are used to mitigate these artifacts. A common filtering approach to using small textures for larger rendering areas is "bilinear" filtering.

Another issue occurs when the area being rendered to is at a very oblique angle to the view (for example, a 256 by 256 texture being rendered to an area 100 pixels wide but only 5 pixels deep because of the viewing angle). In this case "anisotropic" filtering is often applied. Anisotropic filtering provides better image quality than bilinear filtering, as it removes aliasing effects without excessive blurring, but is more computationally expensive.

Related topics

[Views](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Render target view (RTV)

Article • 10/20/2022

Render targets enable a scene to be rendered to a temporary intermediate buffer, rather than to the back buffer to be rendered to the screen. This feature enables use of the complex scene that might be rendered, perhaps as a reflection texture or other purpose within the graphics pipeline, or perhaps to add additional pixel shader effects to the scene before rendering.

Related topics

[Views](#)

Depth stencil view (DSV)

Article • 10/20/2022

A depth stencil view provides the format and buffer for holding depth and stencil information. The depth buffer is used to cull the drawing of pixels that would be invisible to the viewer as they are occluded from view by a closer object. The stencil buffer can be used to cull all drawing outside of a defined shape.

There are a number of more advanced uses of stencil buffers beyond defining a rendering area. Stencil buffer values can be manipulated to give such effects as fades, silhouettes, decaling, dissolves, outlining, shadow volumes, and so on.

Related topics

[Views](#)

Stream output view (SOV)

Article • 10/20/2022

Stream output views enable the vertex information that the vertex, tessellation and geometry shaders have come up with to be streamed back out to the application for further use. For example, an object that has been distorted by these shaders could be written back to the application to provide more accurate input to a physics or other engine. In practice though, stream output views are an infrequently used feature of the graphics pipeline.

Related topics

[Views](#)

Rasterizer ordered view (ROV)

Article • 10/20/2022

Rasterizer ordered views enable some of the limitations of a depth buffer to be addressed, in particular having multiple textures containing transparency all applying to the same pixels.

Rasterizer ordered views enable "Order Independent Transparency" (OIT) algorithms to be applied to the rendering of pixels. A depth buffer only enables a pixel to be drawn or occluded, there is no concept of partial occlusion through transparency. OIT algorithms apply transparent textures in the correct order, so if, for example, a transparent glass object should appear behind a glass window that is behind some vegetation that uses transparent textures, then the final result is drawn predictably correct. Without ROVs and OIT algorithms, the order these transparent objects would be drawn was unpredictable and the rendered scene could simply be confusing and wrong.

Related topics

[Views](#)

Compute pipeline

Article • 10/20/2022

The Direct3D compute pipeline is designed to handle calculations that can be done mostly in parallel with the graphics pipeline. There are only a few steps in the compute pipeline, with data flowing from input to output through the programmable compute shader stage.

Purpose

Like other programmable shaders, [Compute Shader \(CS\) stage](#) is designed and implemented with HLSL. A compute shader provides high-speed general purpose computing and takes advantage of the large numbers of parallel processors on the graphics processing unit (GPU). The compute shader provides memory sharing and thread synchronization features to allow more effective parallel programming methods.

Input

Unlike other programmable shaders, the definition of input is abstract. The input can be one, two or three-dimensional in nature, determining the number of invocations of the compute shader to execute. It is possible to define shared data for one set of invocations to read.

Output

Output data from the compute shader, which can be highly varied, can be synchronized with the graphics rendering pipeline when the computed data is required.

Related topics

[Direct3D Graphics Learning Guide](#)

Compute shader (CS) stage

Article • 10/20/2022

The compute shader (CS) stage provides high-speed general-purpose computing, and takes advantage of the large numbers of parallel processors on the graphics processing unit (GPU). The compute shader stage provides memory sharing and thread synchronization features to allow more effective parallel programming methods.

A compute shader can run on many threads in parallel.

A compute shader is an [HLSL](#)-programmable shader stage that expands Direct3D beyond graphics programming. Compute shader technology is also known as *DirectCompute* technology. For more info, also see [Compute shader overview](#).

Related topics

[Compute pipeline](#) [Graphics pipeline](#)

Resources

Article • 10/20/2022

A resource is an area in memory that can be accessed by the Direct3D pipeline. In order for the pipeline to access memory efficiently, data that is provided to the pipeline (such as input geometry, shader resources, and textures) must be stored in a resource. There are two types of resources from which all Direct3D resources derive: a buffer or a texture. Up to 128 resources can be active for each pipeline stage.

Each application will typically create many resources. Examples of resource include: vertex buffers, index buffer, constant buffer, textures, and shader resources. There are several options that determine how resources can be used. You can create resources that are strongly typed or type less; you can control whether resources have both read and write access; you can make resources accessible to only the CPU, GPU, or both. Naturally, there will be speed vs. functionality tradeoff - the more functionality you allow a resource to have, the less performance you should expect.

Since an application often uses many textures, Direct3D also has the concept of a texture array to simplify texture management. A texture array contains one or more textures (all of the same type and dimensions) that can be indexed from within an application or by shaders. Texture arrays allow you to use a single interface with multiple indexes to access many textures. You can create as many texture arrays to manage different texture types as you need.

Once you have created the resources that your application will use, you connect or bind each resource to the pipeline stages that will use them. This is accomplished by calling a bind API, which takes a pointer to the resource. Since more than one pipeline stage may need access to the same resource, Direct3D has the concept of a resource view. A view identifies the portion of a resource that can be accessed. You can create m views of a resource and bind them to n pipeline stages, assuming you follow binding rules for shared resource (the runtime will generate errors at compile time if you don't).

A resource view provides a general model for access to a resource (such as textures or buffers). Because you can use a view to tell the runtime what data to access and how to access it, resource views allow you create type less resources. That is, you can create resources for a given size at compile time, and then declare the data type within the resource when the resource gets bound to the pipeline. Views expose many capabilities for using resources, such as the ability to read back depth/stencil surfaces in the shader, generating dynamic cubemaps in a single pass, and rendering simultaneously to multiple slices of a volume.

In this section

Topic	Description
Resource types	Different types of resources have a distinct layout (or memory footprint). All resources used by the Direct3D pipeline derive from two basic resource types: buffers and textures . A buffer is a collection of raw data (elements); a texture is a collection of texels (texture elements).
Choosing a resource	A resource is a collection of data that is used by the 3D pipeline. Creating resources and defining their behavior is the first step toward programming your application. This guide covers basic topics for choosing the resources required by your application.
Copying and accessing resource data	Usage flags indicate how the application intends to use the resource data, to place resources in the most performant area of memory possible. Resource data is copied across resources so that the CPU or GPU can access it without impacting performance.
Texture views	In Direct3D, texture resources are accessed with a view, which is a mechanism for hardware interpretation of a resource in memory. A view allows a particular pipeline stage to access only the subresources it needs, in the representation desired by the application.

Related topics

[Coordinate systems](#)

[Direct3D Graphics Learning Guide](#)

[Floating-point rules](#)

[Data type conversion](#)

Resource types

Article • 10/20/2022

Different types of resources have a distinct layout (or memory footprint). All resources used by the Direct3D pipeline derive from two basic resource types: [buffers](#) and [textures](#). A buffer is a collection of raw data (elements); a texture is a collection of texels (texture elements).

There are two ways to fully specify the layout (or memory footprint) of a resource:

Item	Description
Typed	Fully specify the type when the resource is created.
Typeless	Fully specify the type when the resource is bound to the pipeline.

Buffer resources

A buffer resource is a collection of fully typed data; internally, a buffer contains elements. An element is made up of 1 to 4 components. Examples of element data types include: a packed data value (like R8G8B8A8), a single 8-bit integer, four 32-bit float values. These data types are used to store data, such as a position vector, a normal vector, a texture coordinate in a vertex buffer, an index in an index buffer, or device state.

A buffer is created as an unstructured resource. Because it is unstructured, a buffer cannot contain any mipmap levels, is not filtered when read, and cannot be multisampled.

Buffer types

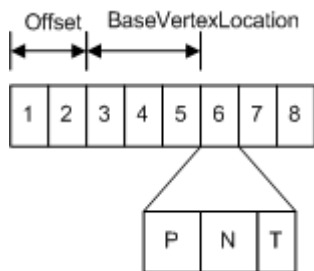
- [Vertex buffer](#)
- [Index buffer](#)
- [Constant buffer](#)

Vertex buffer

A buffer is a collection of elements; a vertex buffer contains per-vertex data. The simplest example is a vertex buffer that contains one type of data, such as position data. It can be visualized like the following illustration.



More often, a vertex buffer contains all the data needed to fully specify 3D vertices. An example of this could be a vertex buffer that contains per-vertex position, normal and texture coordinates. This data is usually organized as sets of per-vertex elements, as shown in the following illustration.



This vertex buffer contains per-vertex data for eight vertices; each vertex stores three elements (position, normal, and texture coordinates). The position and normal are each typically specified using three 32-bit floats and the texture coordinates using two 32-bit floats.

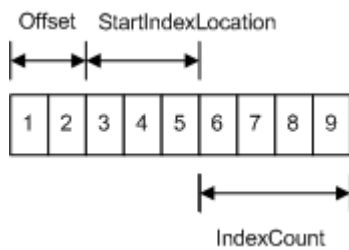
To access data from a vertex buffer, you need to know which vertex to access and these other buffer parameters:

- *Offset* - the number of bytes from the start of the buffer to the data for the first vertex.
- *BaseVertexLocation* - the number of bytes from the offset to the first vertex used by the appropriate draw call.

Before you create a vertex buffer, you need to define its layout by creating an input-layout object. Once the input-layout object is created, bind it to the Input Assembler (IA) stage.

Index buffer

An index buffer contains a sequential set of 16-bit or 32-bit indices; each index is used to identify a vertex in a vertex buffer. Using an index buffer with one or more vertex buffers to supply data to the IA stage is called indexing. An index buffer can be visualized like the following illustration.



The sequential indices stored in an index buffer are located with the following parameters:

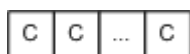
- *Offset* - the number of bytes from the start of the buffer to the first index.
- *StartIndexLocation* - the number of bytes from the offset to the first vertex used by the appropriate draw call.
- *IndexCount* - the number of indices to render.

An index buffer can stitch together multiple line or triangle strips ([primitive topologies](#)) by separating each with a strip-cut index. A strip-cut index allows multiple line or triangle strips to be drawn with a single draw call. A strip-cut index is the maximum possible value for the index (0xffff for a 16-bit index, 0xffffffff for a 32-bit index). The strip-cut index resets the winding order in indexed primitives and can be used to remove the need for degenerate triangles that may otherwise be required to maintain proper winding order in a triangle strip. The following illustration shows an example of a strip-cut index.



Constant buffer

Direct3D has a buffer for supplying shader constants called a shader-constant buffer or simply a constant buffer. Conceptually, it looks like a single-element vertex buffer, as shown in the following illustration.



Each element stores a 1-to-4 component constant, determined by the format of the data stored.

Constant buffers reduce the bandwidth required to update shader constants by allowing shader constants to be grouped together and committed at the same time rather than making individual calls to commit each constant separately.

A shader continues to read variables in a constant buffer directly by variable name in the same manner variables that are not part of a constant buffer are read.

Each shader stage allows up to 15 shader-constant buffers; each buffer can hold up to 4096 constants.

Use a constant buffer to store the results of the stream-output stage.

See [Shader Constants \(DirectX HLSL\)](#) for an example of declaring a constant buffer in a shader.

Texture resources

A texture resource is a structured collection of data designed to store texels. Unlike buffers, textures can be filtered by texture samplers as they are read by shader units. The type of texture impacts how the texture is filtered. A texel represents the smallest unit of a texture that can be read or written to by the pipeline. Each texel contains 1 to 4 components, arranged in one of the DXGI formats (see [DXGI_FORMAT](#)).

Textures are created as a structured resource so that their size is known. However, each texture may be typed or typeless at resource-creation time, as long as the type is fully specified using a view when the texture is bound to the pipeline.

- [Texture types](#)
- [Subresources](#)
- [Strong vs. Weak Typing](#)

Texture types

There are several types of textures: 1D, 2D, 3D, each of which can be created with or without mipmaps. Direct3D also supports texture arrays and multisampled textures.

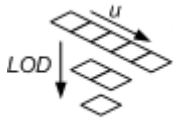
- [1D Texture](#)
- [1D texture array](#)
- [2D Texture and 2D Texture Array](#)
- [3D Texture](#)

1D Texture

A 1D texture in its simplest form contains texture data that can be addressed with a single texture coordinate; it can be visualized as an array of texels, as shown in the following illustration.



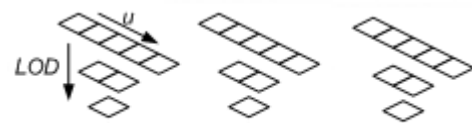
Each texel contains a number of color components depending on the format of the data stored. Adding more complexity, you can create a 1D texture with mipmap levels, as shown in the following illustration.



A mipmap level is a texture that is a power-of-two smaller than the level above it. The top-most level contains the most detail, each subsequent level is smaller; for a 1D mipmap, the smallest level contains one texel. The differing levels are identified by an index called a LOD (level-of-detail); you can use the LOD to access a smaller texture when rendering geometry that is not as close to the camera.

1D texture array

Direct3D 10 also has a new data structure for an array of textures. An array of 1D textures looks conceptually like the following illustration.



This texture array contains three textures. Each of the three textures has a texture width of 5 (which is the number of elements in the first layer). Each texture also contains a 3-layer mipmap.

All texture arrays in Direct3D are a homogeneous array of textures; this means that every texture in a texture array must have the same data format and size (including texture width and number of mipmap levels). You may create texture arrays of different sizes, as long as all the textures in each array match in size.

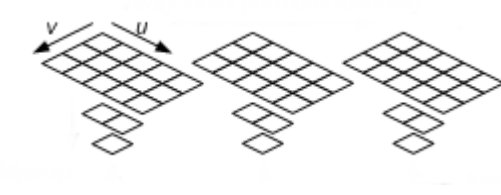
2D Texture and 2D Texture Array

A Texture2D resource contains a 2D grid of texels. Each texel is addressable by a *u*, *v* vector. Because it is a texture resource, it may contain mipmap levels, and subresources. A fully populated 2D texture resource looks like the following illustration.



This texture resource contains a single 3x5 texture with three mipmap levels.

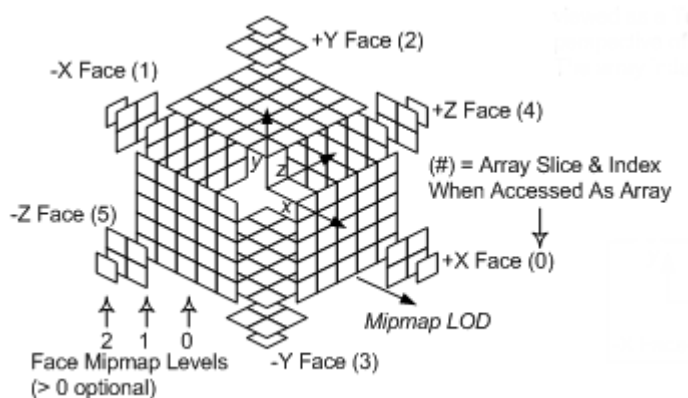
A Texture2DArray resource is a homogeneous array of 2D textures; that is, each texture has the same data format and dimensions (including mipmap levels). It has a similar layout as the 1D texture array except that the textures now contain 2D data, and therefore looks like the following illustration.



This texture array contains three textures; each texture is 3x5 with two mipmap levels.

Using a Texture2DArray as a Texture Cube

A texture cube is a 2D texture array that contains 6 textures, one for each face of the cube. A fully populated texture cube looks like the following illustration.

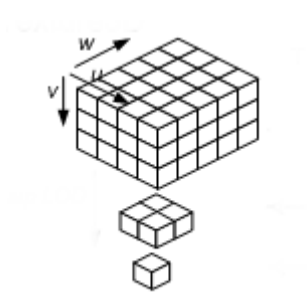


A 2D texture array that contains 6 textures may be read from within shaders with the cube map intrinsic functions, after they are bound to the pipeline with a cube-texture view. Texture cubes are addressed from the shader with a 3D vector pointing out from the center of the texture cube.

3D Texture

A Texture3D resource (also known as a volume texture) contains a 3D volume of texels. Since it is a texture resource, it may contain mipmap levels. A fully populated 3D texture

looks like the following illustration.



When a 3D texture mipmap slice is bound as a render target output (with a render-target view), the 3D texture behaves identically to a 2D texture array with n slices. The particular render slice is chosen from the geometry-shader stage.

There is no concept of a 3D texture array; therefore a 3D texture subresource is a single mipmap level.

Subresources

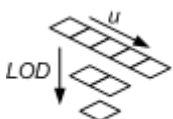
The Direct3D API references entire resources or subsets of resources. To specify portion of resources, Direct3D has coined the term *subresources*, which means a subset of a resource.

A buffer is defined as a single subresource. Textures are a little more complicated as there are several different texture types (1D, 2D, etc.) some of which support mipmap levels and/or texture arrays. Beginning with the simplest case, a 1D texture is defined as a single subresource, as shown in the following illustration.



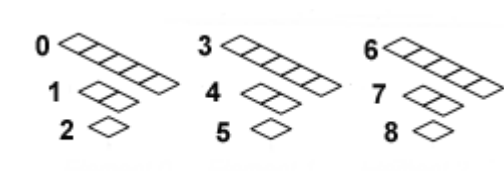
This means that the array of texels that make up a 1D texture are contained in a single subresource.

If you expand a 1D texture with three mipmap levels, it can be visualized like this.



Think of this as a single texture that is made up of three subtextures. Each subtexture is counted as a subresource, so this 1D texture contains 3 subresources. A subtexture (or subresource) can be indexed using the level-of-detail (LOD) for a single texture. When using an array of textures, accessing a particular subtexture requires both the LOD and

the particular texture. Alternately, the API combines these two pieces of information into a single zero-based subresource index as shown here.



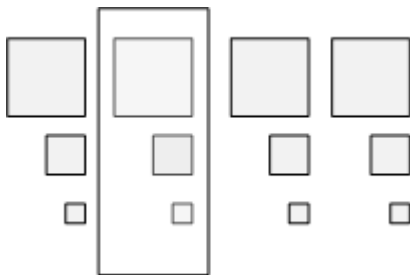
Selecting Subresources

Some APIs access an entire resource, others access a portion of a resource. The APIs that access a portion of a resource generally use a view description to specify the subresources to access.

These figures illustrate the terms used by a view description when accessing an array of textures.

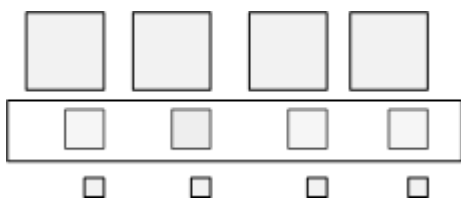
Array Slice

Given an array of textures, each texture with mipmaps, an array slice (represented by the white rectangle) includes one texture and all of its subtextures, as shown in the following illustration.



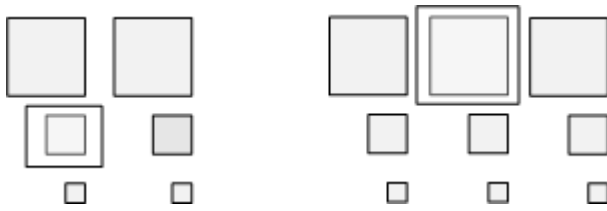
Mip Slice

A mip slice (represented by the white rectangle) includes one mipmap level for every texture in an array, as shown in the following illustration.



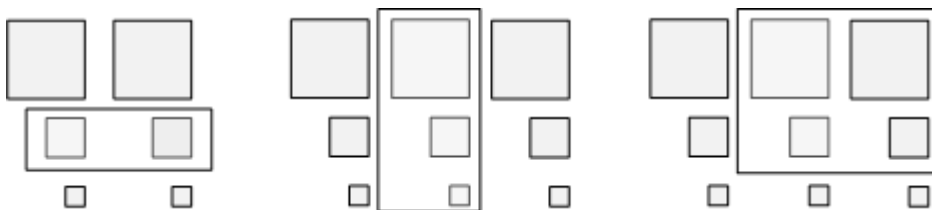
Selecting a Single Subresource

You can use these two types of slices to choose a single subresource, as shown in the following illustration.



Selecting Multiple Subresources

Or you can use these two types of slices with the number of mipmap levels and/or number of textures, to choose multiple subresources.



Regardless of what texture type you are using, with or without mipmaps, with or without a texture array, there are often helper functions provided to compute the index of a particular subresource.

Strong vs. Weak Typing

Creating a fully-typed resource restricts the resource to the format it was created with. This enables the runtime to optimize access, especially if the resource is created with flags indicating that it cannot be mapped by the application. Resources created with a specific type cannot be reinterpreted using the view mechanism.

In a typeless resource, the data type is unknown when the resource is first created. The application must choose from the available typeless formats. You must specify the size of the memory to allocate and whether the runtime will need to generate the subtextures in a mipmap.

However, the exact data format (whether the memory will be interpreted as integers, floating point values, unsigned integers etc.) is not determined until the resource is bound to the pipeline with a view. As the texture format remains flexible until the texture is bound to the pipeline, the resource is referred to as weakly typed storage. Weakly typed storage has the advantage that it can be reused or reinterpreted (in another format) as long as the component bit of the new format matches the bit count of the old format.

A single resource can be bound to multiple pipeline stages as long as each has a unique view, which fully qualifies the formats at each location. For example, a resource created with a typeless format could be used as a FLOAT format and a UINT format at different locations in the pipeline simultaneously.

Related topics

[Resources](#)

Choosing a resource

Article • 10/20/2022

A resource is a collection of data that is used by the 3D pipeline. Creating resources and defining their behavior is the first step toward programming your application. This guide covers basic topics for choosing the resources required by your application.

Identify pipeline stages that need resources

The first step is to choose the [graphics pipeline](#) stage (or stages) that will use a resource. That is, identify each stage that will read data from a resource as well as the stages that will write data out to a resource. Knowing the pipeline stages where the resources will be used determines the APIs that will be called to bind the resource to the stage.

This table lists the types of resources that can be bound to each pipeline stage. It includes whether the resource can be bound as an input or an output.

Pipeline Stage	In/Out	Resource	Resource Type
Input Assembler	In	Vertex Buffer	Buffer
Input Assembler	In	Index Buffer	Buffer
Shader Stages	In	Shader-ResourceView	Buffer, Texture1D, Texture2D, Texture3D
Shader Stages	In	Shader-Constant Buffer	Buffer
Stream Output	Out	Buffer	Buffer
Output Merger	Out	Render-target View	Buffer, Texture1D, Texture2D, Texture3D
Output Merger	Out	Depth/Stencil View	Texture1D, Texture2D

Identify how each resource will be used

Once you have chosen the pipeline stages that your application will use (and therefore the resources that each stage will require), the next step is to determine how each resource will be used, that is, whether a resource can be accessed by the CPU or the GPU.

The hardware that your application is running on will have a minimum of at least one CPU and one GPU. To pick a usage value, consider which type of processor needs to read or write to the resource from the following options.

Resource Usage	Can be updated by	Frequency of Update
Default	GPU	infrequently
Dynamic	CPU	frequently
Staging	GPU	n/a
Immutable	CPU (only at resource creation time)	n/a

Default usage should be used for a resource that is expected to be updated by the CPU infrequently (less than once per frame). Ideally, the CPU would never write directly to a resource with default usage so as to avoid potential performance penalties.

Dynamic usage should be used for a resource that the CPU updates relatively frequently (once or more per frame). A typical scenario for a dynamic resource would be to create dynamic vertex and index buffers that would be filled at runtime with data about the geometry visible from the point of view of the user for each frame. These buffers would be used to render only the geometry visible to the user for that frame.

Staging usage should be used to copy data to and from other resources. A typical scenario would be to copy data in a resource with default usage (which the CPU cannot access) to a resource with staging usage (which the CPU can access).

Immutable resources should be used when the data in the resource will never change.

Another way of looking at the same idea is to think of what an application does with a resource.

How Application uses the Resource	Resource Usage
Load once and never update	Immutable or Default
Application fills resource repeatedly	Dynamic
Render to texture	Default
CPU access of GPU data	Staging

If you are unsure what usage to choose, start with the default usage as it is expected to be the most common case. A Shader-Constant buffer is the one resource type that should always have default usage.

Binding resources to pipeline stages

A resource can be bound to more than one pipeline stage at the same time as long as the restrictions specified when the resource was created are met. These restrictions are specified as usage flags, bind flags, or cpu access flags. More specifically, a resource can be bound as an input and an output simultaneously as long as reading and writing part of a resource cannot happen at the same time.

When binding a resource, think about how the GPU and the CPU will access the resource. Resources that are designed for a single purpose (do not use multiple usage, bind, and cpu access flags) will more than likely result in better performance.

For example, consider the case of a render target used as a texture multiple times. It may be faster to have two resources: a render target and a texture used as a shader resource. Each resource would use only one bind flag, indicating "render target" or "shader resource". The data would be copied from the render-target texture to the shader texture.

This technique in this example may improve performance by isolating the render-target write from the shader-texture read. The only way to be sure is to implement both approaches and measure the performance difference in your particular application.

Related topics

[Resources](#)

Copying and accessing resource data

Article • 10/20/2022

Usage flags indicate how the application intends to use the resource data, to place resources in the most performant area of memory possible. Resource data is copied across resources so that the CPU or GPU can access it without impacting performance.

It isn't necessary to think about resources as being created in either video memory or system memory, or to decide whether or not the runtime should manage the memory. With the architecture of the WDDM (Windows Display Driver Model), applications create Direct3D resources with different usage flags to indicate how the application intends to use the resource data. This driver model virtualizes the memory used by resources; it is the responsibility of the operating system/driver/memory manager to place resources in the most performant area of memory possible, given the expected usage.

The default case is for resources to be available to the GPU. There are times when the resource data needs to be available to the CPU. Copying resource data around so that the appropriate processor can access it without impacting performance requires some knowledge of how the API methods work.

Copying resource data

Resources are created in memory when Direct3D executes a Create call. They can be created in video memory, system memory, or any other kind of memory. Since WDDM driver model virtualizes this memory, applications no longer need to keep track of what kind of memory resources are created in.

Ideally, all resources would be located in video memory so that the GPU can have immediate access to them. However, it is sometimes necessary for the CPU to read the resource data or for the GPU to access resource data the CPU has written to. Direct3D handles these different scenarios by requesting the application specify a usage, and then offers several methods for copying resource data when necessary.

Depending on how the resource was created, it is not always possible to directly access the underlying data. This may mean that the resource data must be copied from the source resource to another resource that is accessible by the appropriate processor. In terms of Direct3D, default resources can be accessed directly by the GPU, dynamic and staging resources can be directly accessed by the CPU.

Once a resource has been created, its usage cannot be changed. Instead, copy the contents of one resource to another resource that was created with a different usage.

You copy resource data from one resource to another, or copy data from memory to a resource.

There are two main kinds of resources: mappable and non-mappable. Resources created with dynamic or staging usages are mappable, while resources created with default or immutable usages are non-mappable.

Copying data among non-mappable resources is very fast because this is the most common case and has been optimized to perform well. Since these resources are not directly accessible by the CPU, they are optimized so that the GPU can manipulate them quickly.

Copying data among mappable resources is more problematic because the performance will depend on the usage the resource was created with. For example, the GPU can read a dynamic resource fairly quickly but cannot write to them, and the GPU cannot read or write to staging resources directly.

Applications that wish to copy data from a resource with default usage to a resource with staging usage (to allow the CPU to read the data -- that is, the GPU readback problem) must do so with care. See [Accessing resource data](#), below.

Accessing resource data

Accessing a resource requires mapping the resource; mapping essentially means the application is trying to give the CPU access to memory. The process of mapping a resource so that the CPU can access the underlying memory can cause some performance bottlenecks and for this reason, care must be taken as to how and when to perform this task.

Performance can grind to a halt if the application tries to map a resource at the wrong time. If the application tries to access the results of an operation before that operation is finished, a pipeline stall will occur.

Performing a map operation at the wrong time could potentially cause a severe drop in performance by forcing the GPU and the CPU to synchronize with each other. This synchronization will occur if the application wants to access a resource before the GPU is finished copying it into a resource the CPU can map.

Performance considerations

It is best to think of a PC as a machine running as a parallel architecture with two main types of processors: one or more CPU's and one or more GPU's. As in any parallel

architecture, the best performance is achieved when each processor is scheduled with enough tasks to prevent it from going idle and when the work of one processor is not waiting on the work of another.

The worst-case scenario for GPU/CPU parallelism is the need to force one processor to wait for the results of work done by another. Direct3D removes this cost by making the copy methods asynchronous; the copy has not necessarily executed by the time the method returns.

The benefit of this is that the application does not pay the performance cost of actually copying the data until the CPU accesses the data, which is when Map is called. If the Map method is called after the data has actually been copied, no performance loss occurs. On the other hand, if the Map method is called before the data has been copied, then a pipeline stall will occur.

Asynchronous calls in Direct3D (which are the vast majority of methods, and especially rendering calls) are stored in what is called a *command buffer*. This buffer is internal to the graphics driver and is used to batch calls to the underlying hardware so that the costly switch from user mode to kernel mode in Microsoft Windows occurs as rarely as possible.

The command buffer is flushed, thus causing a user/kernel mode switch, in one of four situations, which are as follows.

1. Present is called.
2. Flush is called.
3. The command buffer is full; its size is dynamic and is controlled by the Operating System and the graphics driver.
4. The CPU requires access to the results of a command waiting to execute in the command buffer.

Of the four situations above, number four is the most critical to performance. If the application issues a call to copy a resource, or subresource, this call is queued in the command buffer.

If the application then tries to map the staging resource that was the target of the copy call before the command buffer has been flushed, a pipeline stall will occur, because not only does the Copy method call need to execute, but all other buffered commands in the command buffer must execute as well. This will cause the GPU and CPU to synchronize because the CPU will be waiting to access the staging resource while the GPU is emptying the command buffer and finally filling the resource the CPU needs. Once the GPU finishes the copy, the CPU will begin accessing the staging resource, but during this time, the GPU will be sitting idle.

Doing this frequently at runtime will severely degrade performance. For that reason, mapping of resources created with default usage should be done with care. The application needs to wait long enough for the command buffer to be emptied and thus have all of those commands finish executing before it tries to map the corresponding staging resource.

How long should the application wait? At least two frames because this will enable parallelism between the CPU(s) and the GPU to be maximally leveraged. The way the GPU works is that while the application is processing frame N by submitting calls to the command buffer, the GPU is busy executing the calls from the previous frame, N-1.

So if an application wants to map a resource that originates in video memory and copies a resource at frame N, this call will actually begin to execute at frame N+1, when the application is submitting calls for the next frame. The copy should be finished when the application is processing frame N+2.

Frame	GPU/CPU Status
N	• CPU issues render calls for current frame.
N+1	• GPU executing calls sent from CPU during frame N. • CPU issues render calls for current frame.
N+2	• GPU finished executing calls sent from CPU during frame N. Results ready. • GPU executing calls sent from CPU during frame N+1. • CPU issues render calls for current frame.
N+3	• GPU finished executing calls sent from CPU during frame N+1. Results ready. • GPU executing calls sent from CPU during frame N+2. • CPU issues render calls for current frame.
N+4	...

Related topics

[Resources](#)

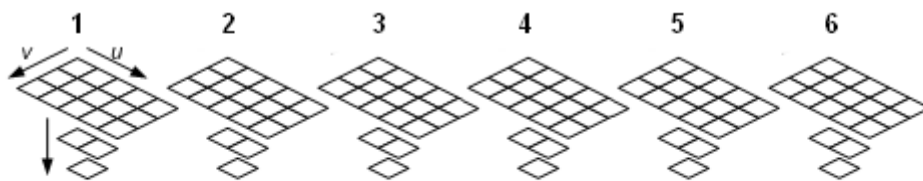
Texture views

Article • 10/20/2022

In Direct3D, texture resources are accessed with a view, which is a mechanism for hardware interpretation of a resource in memory. A view allows a particular pipeline stage to access only the [subresources](#) it needs, in the representation desired by the application.

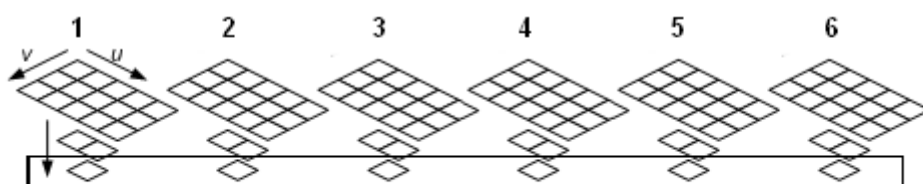
A view supports the notion of a typeless resource. A typeless resource is a resource created with a specific size but not a specific data type. The data is interpreted dynamically when it is bound to the pipeline.

The following illustration shows an example of binding a 2D texture array with 6 textures as a shader resource by creating a shader resource view for it. The resource is then addressed as an array of textures. (Note: a subresource cannot be bound as both input and output to the pipeline simultaneously.)



When using a 2D texture array as a render target, the resource can be viewed as an array of 2D textures (6 in this example) with mipmap levels (3 in this example).

Create a view object for a render target by calling `CreateRenderTargetView`. Then call `OMSetRenderTargets` to set the render target view to the pipeline. Render into the render targets by calling `Draw` and using the `RenderTargetArrayIndex` to index into the proper texture in the array. You can use a subresource (a mipmap level, array index combination) to bind to any array of subresources. So you could bind to the second mipmap level and only update this particular mipmap level if you wanted, as in the following illustration.



Related topics

[Resources](#)

Streaming resources

Article • 10/20/2022

Streaming resources are large logical resources that use small amounts of physical memory. Instead of passing an entire large resource, small parts of the resource are streamed as needed. Streaming resources were previously called *tiled resources*.

In this section

Topic	Description
The need for streaming resources	Streaming resources are needed so GPU memory isn't wasted storing regions of surfaces that won't be accessed, and to tell the hardware how to filter across adjacent tiles.
Creating streaming resources	Streaming resources are created by specifying a flag when you create a resource, indicating that the resource is a streaming resource.
Pipeline access to streaming resources	Streaming resources can be used in shader resource views (SRV), render target views (RTV), depth stencil views (DSV) and unordered access views (UAV), as well as some bind points where views aren't used, such as vertex buffer bindings.
Streaming resources features tiers	Direct3D supports streaming resources in three tiers of capabilities.

Related topics

[Direct3D Graphics Learning Guide](#)

[Resources](#)

The need for streaming resources

Article • 10/20/2022

Streaming resources are needed so GPU memory isn't wasted storing regions of surfaces that won't be accessed, and to tell the hardware how to filter across adjacent tiles.

Streaming resources or sparse textures

Streaming resources (called *tiled resources* in Direct3D 11), are large logical resources that use small amounts of physical memory.

Another name for streaming resources is *sparse textures*. "Sparse" conveys both the tiled nature of the resources as well as perhaps the primary reason for tiling them - that not all of them are expected to be mapped at once. In fact, an application could conceivably author a streaming resource in which no data is authored for all regions+mips of the resource, intentionally. So, the content itself could be sparse, and the mapping of the content in graphics processing unit (GPU) memory at a given time would be a subset of that (even more sparse).

Without tiling, memory allocations are managed at subresource granularity

In a graphics system (that is, the operating system, display driver, and graphics hardware) without streaming resource support, the graphics system manages all Direct3D memory allocations at subresource granularity.

For a [buffer](#), the entire buffer is the subresource.

For a [Texture](#), (for example, [Texture2D](#)), each mip level is a subresource; for a texture array, (for example, [Texture2DArray](#)) each mip level at a given array slice is a subresource. The graphics system only exposes the ability to manage the mapping of allocations at this subresource granularity. In the context of streaming resources, "mapping" refers to making data visible to the GPU.

Without tiling, can't access only a small portion of mipmap chain

Suppose an application knows that a particular rendering operation only needs to access a small portion of an image mipmap chain (perhaps not even the full area of a given mipmap). Ideally, the app could inform the graphics system about this need. The graphics system would then only bother to ensure that the needed memory is mapped on the GPU without paging in too much memory.

In reality, without streaming resource support, the graphics system can only be informed about the memory that needs to be mapped on the GPU at subresource granularity (for example, a range of full mipmap levels that could be accessed). There is no demand faulting in the graphics system either, so potentially a lot of excess GPU memory must be used to make full subresources mapped before a rendering command that references any part of the memory is executed. This is just one issue that makes the use of large memory allocations difficult in Direct3D without streaming resource support.

Software paging to break the surface into smaller tiles

Software paging can be used to break the surface into tiles that are small enough for the hardware to handle.

Direct3D supports [Texture2D](#) surfaces with up to 16384 pixels on a given side. An image that is 16384 wide by 16384 tall and 4 bytes per pixel would consume 1GB of video memory (and adding mipmaps would double that amount). In practice, all 1GB would rarely need to be referenced in a single rendering operation.

Some game developers model terrain surfaces as large as 128K by 128K. The way they get this to work on existing GPUs is to break the surface into tiles that are small enough for hardware to handle. The application must figure out which tiles might be needed and load them into a cache of textures on the GPU - a software paging system.

A significant downside to that approach comes from the hardware not knowing anything about the paging that is going on: When a part of an image needs to be shown on screen that straddles tiles, the hardware does not know how to perform fixed function (that is, efficient) filtering across tiles. This means the application managing its own software tiling must resort to manual texture filtering in shader code (which becomes very expensive if a good quality anisotropic filter is desired) and/or waste memory authoring gutters around tiles that contain data from neighboring tiles so that fixed function hardware filtering can continue to provide some assistance.

Making tiled representation of surface allocations a first-class feature

If a tiled representation of surface allocations is a first-class feature in the graphics system, the application could tell the hardware which tiles to make available. In this way, less GPU memory is wasted storing regions of surfaces that the application knows will not be accessed, and the hardware can understand how to filter across adjacent tiles, alleviating some of the pain experienced by developers who perform software tiling on their own.

But to provide a complete solution, something must be done to deal with the fact that, independent of whether tiling within a surface is supported, the maximum surface dimension is currently 16384 - nowhere near the 128K+ that applications already want. Just requiring the hardware to support larger texture sizes is one approach, however there are significant costs and/or tradeoffs to going this route.

Direct3D's texture filter path and rendering path are already saturated in terms of precision in supporting 16K textures with the other requirements, such as supporting viewport extents falling off the surface during rendering, or supporting texture wrapping off the surface edge during filtering. A possibility is to define a tradeoff such that as the texture size increases beyond 16K, functionality/precision is given up in some manner. Even with this concession however, additional hardware costs might be required in terms of addressing capability throughout the hardware system to go to larger texture sizes.

Issue with large textures: precision for locations on surface

One issue that comes into play as textures get very large is that single precision floating point texture coordinates (and the associated interpolators to support rasterization) run out of precision to specify locations on the surface accurately. Jittery texture filtering would ensue. One expensive option would be to require double precision interpolator support, though that could be overkill given a reasonable alternative.

Enabling multiple resources of different dimensions to share memory

Another scenario that could be served by streaming resources is enabling multiple resources of different dimensions/formats to share the same memory. Sometimes

applications have exclusive sets of resources that are known not to be used at the same time, or resources that are created only for very brief use and then destroyed, followed by creation of other resources. A form of generality that can fall out of "streaming resources" is that it is possible to allow the user to point multiple different resources at the same (overlapping) memory. In other words, the creation and destruction of "resources" (which define a dimension/format and so on) can be decoupled from the management of the memory underlying the resources from the application's point of view.

Related topics

[Streaming resources](#)

Creating streaming resources

Article • 10/20/2022

Streaming resources are created by specifying a flag when you create a resource, indicating that the resource is a streaming resource.

Restrictions on when you can create a resource as a streaming resource are described in [Streaming resource creation parameters](#).

A non-streaming resource's storage is allocated in the graphics system when the resource is created, such as allocation for an array of 2D textures.

When a streaming resource is created, the graphics system doesn't allocate the storage for the resource contents. Instead, when an application creates a streaming resource, the graphics system makes an address space reservation for the tiled surface's area only, and then allows the mapping of the tiles to be controlled by the application. The "mapping" of a tile is simply the physical location in memory that a logical tile in a resource points to (or **NULL** for an unmapped tile).

Don't confuse this concept with the notion of mapping a Direct3D resource for CPU access, which despite using the same name is completely independent. You will be able to define and change the mapping of each tile individually as needed, knowing that all tiles for a surface don't need to be mapped at a time, thereby making effective use of the amount of memory available.

In this section

Topic	Description
Mappings are into a tile pool	When a resource is created as a streaming resource, the tiles that make up the resource come from pointing at locations in a tile pool. A tile pool is a pool of memory (backed by one or more allocations behind the scenes - unseen by the application).
Streaming resource creation parameters	There are some constraints on the type of Direct3D resources that you can create as a streaming resource.
Tile pool creation parameters	Use the parameters in this section to define tile pools when creating a buffer.

Topic	Description
Streaming resource cross-process and device sharing	Tile pools can be shared with other processes just like traditional resources. Streaming resources that reference tile pools can't be shared across devices and processes.
Operations available on streaming resources	This section lists operations that you can perform on streaming resources.
Operations available on tile pools	Operations on tile pools include resizing a tile pool, offering resources (yielding memory temporarily to the system for the entire tile pool), and reclaiming resources.
How a streaming resource's area is tiled	When you create a streaming resource, the dimensions, format element size, and number of mipmaps and/or array slices (if applicable) determine the number of tiles that are required to back the entire surface area.

Related topics

[Streaming resources](#)

Mappings are into a tile pool

Article • 10/20/2022

When a resource is created as a streaming resource, the tiles that make up the resource come from pointing at locations in a tile pool. A tile pool is a pool of memory (backed by one or more allocations behind the scenes - unseen by the application). The operating system and display driver manage this pool of memory, and the memory footprint is easily understood by an application. Streaming resources map 64KB regions by pointing to locations in a tile pool. One fallout of this setup is it allows multiple resources to share and reuse the same tiles, and also for the same tiles to be reused at different locations within a resource if desired.

The cost for the flexibility of populating the tiles for a resource out of a tile pool is that the resource has to do the work of defining and maintaining the mapping of which tiles in the tile pool represent the tiles needed for the resource. Tile mappings can be changed. Also, not all tiles in a resource need to be mapped at a time; a resource can have **NULL** mappings. A **NULL** mapping defines a tile as not being available from the point of view of the resource accessing it.

Multiple tile pools can be created, and any number of streaming resources can map into any given tile pool at the same time. Tile pools can also be grown or shrunk. For more info, see [Tile pool resizing](#). One constraint that exists to simplify display driver and runtime implementation is that a given streaming resource can only have mappings into at most one tile pool at a time (as opposed to having simultaneous mapping to multiple tile pools).

The amount of storage that is associated with a streaming resource itself (that is, independent tile-pool memory) is roughly proportional to the number of tiles actually mapped to the pool at any given time. In hardware, this fact boils down to scaling the memory footprint for page table storage roughly with the amount of tiles that are mapped (for example, using a multilevel page table scheme as appropriate).

The tile pool can be thought of as an entirely software abstraction that enables Direct3D applications to effectively be able to program the page tables on the graphics processing unit (GPU) without having to know the low level implementation details (or deal with pointer addresses directly). Tile pools don't apply any additional levels of indirection in hardware. Optimizations of a single level page table using constructs like page directories are independent of the tile pool concept.

Let us explore what storage the page table itself could require in the worst case (though in practice implementations only require storage roughly proportional to what is

mapped).

Suppose each page table entry is 64 bits.

For the worst-case page table size hit for a single surface, given the resource limits in Direct3D 11, suppose a streaming resource is created with a 128 bit-per-element format (for example, an RGBA float), so a 64KB tile contains only 4096 pixels. The maximum supported [Texture2DArray](#) size of 16384*16384*2048 (but with only a single mipmap) would require about 1GB of storage in the page table if fully populated (not including mipmaps) using 64 bit table entries. Adding mipmaps would grow the fully-mapped (worst case) page table storage by about a third, to about 1.3GB.

This case would give access to about 10.6 terabytes of addressable memory. There might be a limit on the amount of addressable memory however, which would reduce these amounts, perhaps to around the terabyte range.

Another case to consider is a single [Texture2D](#) streaming resource of 16384*16384 with a 32 bit-per-element format, including mipmaps. The space needed in a fully populated page table would be roughly 170KB with 64 bit table entries.

Finally, consider an example using a BC format, say BC7 with 128 bits per tile of 4x4 pixels. That is one byte per pixel. A [Texture2DArray](#) of 16384*16384*2048 including mipmaps would require roughly 85MB to fully populate this memory in a page table. That is not bad considering this allows one streaming resource to span 550 gigapixels (512 GB of memory in this case).

In practice, nowhere near these full mappings would be defined given that the amount of physical memory available wouldn't allow anywhere near that much to be mapped and referenced at a time. But with a tile pool, applications could choose to reuse tiles (as a simple example, reusing a "black" colored tile for large black regions in an image) - effectively using the tile pool (that is, page table mappings) as a tool for memory compression.

The initial contents of the page table are **NULL** for all entries. Applications also can't pass initial data for the memory contents of the surface since it starts off with no memory backing.

In this section

Topic	Description
Tile pool creation	Applications can create one or more tile pools per Direct3D device. The total size of each tile pool is restricted to Direct3D 11's resource size limit, which is roughly 1/4 of GPU RAM.
Tile pool resizing	Resize a tile pool to grow a tile pool if the application needs more working set for the streaming resources mapping into it, or to shrink if less space is needed.
Hazard tracking versus tile pool resources	For non-streaming resources, Direct3D can prevent certain hazard conditions during rendering, but because hazard tracking would be at a tile level for streaming resources, tracking hazard conditions during rendering of streaming resources might be too expensive.

Related topics

[Creating streaming resources](#)

Feedback

Was this page helpful?

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Tile pool creation

Article • 10/20/2022

Applications can create one or more tile pools per Direct3D device. The total size of each tile pool is restricted to Direct3D 11's resource size limit, which is roughly 1/4 of graphics processing unit (GPU) RAM.

A tile pool is made of 64KB tiles, but the operating system (display driver) manages the entire pool as one or more allocations behind the scenes—the breakdown is not visible to applications. Streaming resources define content by pointing at tiles within a tile pool. Unmapping a tile from a streaming resource is done by pointing the tile to **NULL**. Such unmapped tiles have rules about the behavior of reads or writes; see [Hazard tracking versus tile pool resources](#).

Related topics

[Mappings are into a tile pool](#)

Tile pool resizing

Article • 10/20/2022

Resize a tile pool to grow a tile pool if the application needs more working set for the streaming resources mapping into it, or to shrink if less space is needed. Another option for applications is to allocate additional tile pools for new streaming resources. But if any single streaming resource needs more space than initially available in its tile pool, growing the tile pool is a good option. A streaming resource can't have mappings into multiple tile pools at the same time.

When a tile pool is grown, additional tiles are added to the end via one or more new allocations by the display driver. This breakdown into allocations isn't visible to the application. Existing memory in the tile pool is left untouched, and existing streaming resource mappings into that memory remain intact.

When a tile pool is shrunk, tiles are removed from the end. Tiles are removed even below the initial allocation size, down to 0, which means new mappings can't be made past the new size. But, existing mappings past the end of the new size remain intact and useable. The display driver will keep the memory around as long as mappings to any part of the allocations that the driver uses for the tile pool memory remains. If after shrinking some memory has been kept alive because tile mappings are pointing to it and then the tile pool is regrown again (by any amount), the existing memory is reused first before any additional allocations occur to service the size of the grow operation.

To be able to save memory, an application has to not only shrink a tile pool but also remove/remap existing mappings past the end of the new smaller tile pool size.

The act of shrinking (and removing mappings) doesn't necessarily produce immediate memory savings. Freeing of memory depends on how granular the display driver's underlying allocations for the tile pool are. When shrinking happens to be enough to make a display driver allocation unused, the display driver can free it. If a tile pool was grown, shrinking to previous sizes (and removing/remapping tile mappings correspondingly) is most likely to yield memory savings, though not guaranteed in the case that the sizes don't exactly align with the underlying allocation sizes chosen by the display driver.

Related topics

[Mappings are into a tile pool](#)

Hazard tracking versus tile pool resources

Article • 10/20/2022

For non-streaming resources, Direct3D can prevent certain hazard conditions during rendering, but because hazard tracking would be at a tile level for streaming resources, tracking hazard conditions during rendering of streaming resources might be too expensive.

For example, for non-streaming resources, the runtime doesn't allow any given SubResource to be bound as an input (such as a Shader Resource View) and as an output (such as a Render Target View) at the same time. If such a case is encountered, the runtime unbinds the input. This tracking overhead in the runtime is cheap and is done at the subresource level. One of the benefits of this tracking overhead is to minimize the chances of applications accidentally depending on hardware shader execution order. Hardware shader execution order can vary if not on a given graphics processing unit (GPU), then certainly across different GPUs.

Tracking how resources are bound might be too expensive for streaming resources because tracking is at a tile level. New issues arise such as possibly validating away attempts to render to a Render Target View with one tile mapped to multiple areas in the surface simultaneously. If it turns out this per-tile hazard tracking is too expensive for the runtime, ideally this would at least be an option in the debug layer.

An application must inform the display driver when it has issued a write or read operation to a streaming resource that references tile pool memory that will also be referenced by separate streaming resources in upcoming read or write operations that it is expecting the first operation to complete before the following operations can begin.

Related topics

[Mappings are into a tile pool](#)

Streaming resource creation parameters

Article • 10/20/2022

There are some constraints on the type of Direct3D resources that you can create as a streaming resource.

Supported Resource Type

Texture2D[Array] (including TextureCube[Array], which is a variant of Texture2D[Array]) or Buffer.

****NOT supported:** ****Texture1D[Array]** .

Supported Resource Usage

Default usage.

****NOT supported:** ****Dynamic, staging, or immutable.**

Supported Resource Misc Flags

Tiled; that is, streaming (by definition), texture cube, draw indirect arguments, buffer allow raw views, structured buffer, resource clamp, or generate mips.

****NOT supported:** ****shared, shared keyed mutex, GDI compatible, shared NT handle, restricted content, restrict shared resource, restrict shared resource driver, guarded, or tile pool.**

Supported Bind Flags

Bind as shader resource, render target, depth stencil, or unordered access.

****NOT supported:** ****Bind as constant buffer, vertex buffer (binding a tiled Buffer as an SRV/UAV/RTV is supported), index buffer, stream output, decoder, or video encoder.**

Supported Formats

All formats that would be available for the given configuration regardless of it being tiled, with some exceptions.

Supported Sample Description (Multisample count, quality)

Whatever would be supported for the given configuration regardless of it being tiled, with some exceptions.

Supported Width/Height/MipLevels/ArraySize

Full extents supported by Direct3D. Streaming resources don't have the restriction on total memory size imposed on non-streaming resources. Streaming resources are only constrained by overall virtual address space limits. See [Address space available for streaming resources](#).

The initial contents of tile pool memory are undefined.

In this section

Topic	Description
Address space available for streaming resources	This section specifies the virtual address space that is available for streaming resources.

Related topics

[Creating streaming resources](#)

Address space available for streaming resources

Article • 10/20/2022

This section specifies the virtual address space that is available for streaming resources.

On 64-bit operating systems, at least 40 bits of virtual address space (1 Terabyte) is available.

For 32-bit operating systems, the address space is 32 bit (4 GB). For 32-bit Arm systems, individual streaming resource creation can fail if the allocation would use more than 27 bits of address space (128 MB). This includes any hidden padding in the address space the hardware may use for mipmaps, packed tile padding, and possibly padding surface dimensions to powers of 2.

On graphics systems with a separate page table for the graphics processing unit (GPU), most of this address space will be available to GPU resources made by the application, though GPU allocations made by the display driver fit in the same space.

On future systems with a page table shared between the CPU and GPU, the available address space is shared between all CPU and GPU allocations in a process.

Related topics

[Streaming resource creation parameters](#)

Tile pool creation parameters

Article • 10/20/2022

Use the parameters in this section to define tile pools when creating a buffer.

Size

Allocation size, as a multiple of 64KB. 0 is valid because you can later resize the tile pool.

Supported Resource Misc Flags

Tile pool (identifies the resource as a tile pool), miscellaneous shared resource, shared keyed mutex, or shared NT handle.

Supported Resource Usage

Default usage, only.

Related topics

[Creating streaming resources](#)

Streaming resource cross-process and device sharing

Article • 10/20/2022

Tile pools can be shared with other processes just like traditional resources. Streaming resources that reference tile pools can't be shared across devices and processes. But separate processes can create their own streaming resources that map to tile pools that are shared between those streaming resources.

Shared tile pools can't be resized.

In this section

Topic	Description
Stencil formats not supported with streaming resources	Formats that contain stencil aren't supported with streaming resources.

Related topics

[Creating streaming resources](#)

Stencil formats not supported with streaming resources

Article • 10/20/2022

Formats that contain stencil aren't supported with streaming resources.

Formats that contain stencil include DXGI_FORMAT_D24_UNORM_S8_UINT (and related formats in the R24G8 family) and DXGI_FORMAT_D32_FLOAT_S8X24_UINT (and related formats in the R32G8X24 family).

Some implementations store depth and stencil in separate allocations while others store them together. Tile management for the two schemes would have to be different, and no single API can abstract or rationalize the differences. We recommend for future hardware to support independent depth and stencil surfaces, each independently tiled.

32-bit depth would have 128x128 tiles, and 8-bit stencil would have 256x256 tiles. Therefore, applications would have to live with tile shape misalignment between depth and stencil. But the same problem exists with different render target surface formats already.

Related topics

[Streaming resource cross-process and device sharing](#)

Operations available on streaming resources

Article • 10/20/2022

This section lists operations that you can perform on streaming resources.

- Update Tile Mappings operations that return void, and Copy Tile Mappings operations that return void - These operations point tile locations in a streaming resource to locations in tile pools, or to NULL, or to both. These operations can update a disjoint subset of the tile pointers.
- Copy and Update operations - All the APIs that can copy data to and from a default pool surface work for streaming resources. Reading from unmapped tiles produces 0 and writes to unmapped tiles are dropped.
- Copy Tiles and Update Tiles operations - These operations exist for copying tiles at 64KB granularity to and from any streaming resource and a buffer resource in a canonical memory layout. The display driver and hardware perform any memory "swizzling" necessary for the streaming resource.
- Direct3D pipeline bindings and view creations / bindings that would work on non-streaming resources work on streaming resources as well.

Tile controls are available on immediate or deferred contexts (just like updates to typical resources) and upon execution impact subsequent accesses to the tiles (not previously submitted operations).

Related topics

[Creating streaming resources](#)

Operations available on tile pools

Article • 10/20/2022

Operations on tile pools include resizing a tile pool, offering resources (yielding memory temporarily to the system for the entire tile pool), and reclaiming resources.

- The lifetime of tile pools works like any other Direct3D Resource, backed by reference counting, including in this case tracking of mappings from streaming resources. When the application no longer references a tile pool and any tile mappings to the memory are gone and graphics processing unit (GPU) accesses completed, the operating system will deallocate the tile pool.
- APIs related to surface sharing and synchronization work for tile pools (but not directly on streaming resources). Similar to the behavior for offered tile pools, Direct3D commands that access streaming resources that point to a tile pool are dropped if the tile pool has been shared and is currently acquired by another device and process.
- Resizing a tile pool.
- Offering resources and reclaiming resources - These operations for yielding memory temporarily to the system operate on the entire tile pool (and aren't available for individual streaming resources). If a streaming resource points to a tile in an offered tile pool, the streaming resource behaves as if it is offered (for example, the runtime drops commands that reference it).

Data can't be copied to and from tile pool memory directly. Accesses to the memory are always done through streaming resources.

Related topics

[Creating streaming resources](#)

How a streaming resource's area is tiled

Article • 10/20/2022

When you create a streaming resource, the dimensions, format element size, and number of mipmaps and/or array slices (if applicable) determine the number of tiles that are required to back the entire surface area. The pixel/byte layout within tiles is determined by the implementation. The number of pixels that fit in a tile, depending on the format element size, is fixed and identical whether you use a standard swizzle or not.

The number of tiles that will be used by a given surface size and format element width is well defined and predictable based on the tables in the following sections. For resources that contain mipmaps or cases where surface dimensions don't fill a tile, some constraints exist; see [Mipmap packing](#).

Different streaming resources can point to identical memory with different formats as long as applications don't rely on the results of writing to the memory with one format and reading with another. But applications can rely on the results of writing to the memory with one format and reading with another if the formats are in the same format family (that is, they have the same typeless parent format). For example, `DXGI_FORMAT_R8G8B8A8_UNORM` and `DXGI_FORMAT_R8G8B8A8_UINT` are compatible with each other but not with `DXGI_FORMAT_R16G16_UNORM`.

An exception is where bleeding data from one format aliasing to another is well defined: if a tile completely contains 0 for all its bits, that tile can be used with any format that interprets those memory contents as 0 (regardless of memory layout). So, a tile could be cleared to 0x00 with the format `DXGI_FORMAT_R8_UNORM` and then used with a format like `DXGI_FORMAT_R32G32_FLOAT` and it would appear the contents are still (0.0f,0.0f).

The layout of data within a tile doesn't depend on where the tile is mapped in a resource overall. So, for example, a tile can be reused in different locations of a surface at once with consistent behavior in all locations.

In this section

Topic	Description
Texture2D and Texture2DArray subresource tiling	These tables show how Texture2D and Texture2DArray subresources are tiled.
Texture3D subresource tiling	This table shows how Texture3D subresources are tiled.

Topic	Description
Buffer tiling	A Buffer resource is divided into 64KB tiles, with some empty space in the last tile if the size is not a multiple of 64KB.
Mipmap packing	Some number of mips (per array slice) can be packed into some number of tiles, depending on a streaming resource's dimensions, format, number of mipmaps, and array slices.

Related topics

[Creating streaming resources](#)

Texture2D and Texture2DArray subresource tiling

Article • 10/20/2022

These tables show how [Texture2D](#) and [Texture2DArray](#) subresources are tiled. The values in these tables don't count tail mip packing.

Subresources with multisample counts of 1

This table shows how [Texture2D](#) and [Texture2DArray](#) subresources with multisample counts of 1 are tiled.

Bits/Pixel (1 sample/pixel)	Tile Dimensions (Pixels, WxH)
8	256x256
16	256x128
32	128x128
64	128x64
128	64x64
BC1,4	512x256
BC2,3,5,6,7	256x256

Format bit counts not supported with streaming resources are 96 bpp formats, video formats, DXGI_FORMAT_R1_UNORM, DXGI_FORMAT_R8G8_B8G8_UNORM, and DXGI_FORMAT_R8R8_G8B8_UNORM.

Subresources with various multisample counts

This table shows how [Texture2D](#) and [Texture2DArray](#) subresources with various multisample counts are tiled.

Bits/Pixel (1 sample/pixel)	Tile Dimensions (Pixels, WxH)
1	1x1

Bits/Pixel (1 sample/pixel)	Tile Dimensions (Pixels, WxH)
2	2x1
4	2x2
8	4x2
16	4x4

Only sample counts 1 and 4 are required (and allowed) to be supported with streaming resources. Streaming resources don't currently support 2, 8, and 16 even though they are shown.

Implementations can choose to support 2, 8, and/or 16 sample multisample antialiasing (MSAA) mode for non-streaming resources even though streaming resource don't support them.

Streaming resources with sample counts larger than 1 can't use 128 bpp formats.

The constraints on supported sample counts and formats are due to hardware inconsistencies from the specification.

Related topics

[How a streaming resource's area is tiled](#)